Supervised Machine Learning

Python Programming for Business Intelligence

Sippo Rossi
April 29th 2026

Learning goals

Theoretical:

- Understanding the basic principles of machine learning and the key concepts and processes related to it
- Know the different steps in building a machine learning model and understand why they are needed

Practical:

- Be able to describe what is machine learning
- Be able to describe what is a supervised machine learning problem
- Know how to preprocess data for machine learning
- Know how to use basic supervised machine learning methods

Motivation

Machine learning is being used for many business applications such as pricing, demand forecasting, fraud detection…

While generative AI dominates discussions nowadays, a broader family of older machine learning methods powers most real-world applications in industry and research

Today we focus on supervised learning, the foundation on which much of applied ML, including modern generative systems, is built

Agenda

- What is and why use machine learning
- Challenges of machine learning
- Preprocessing data for machine learning
- Supervised machine learning
    - Train test split
    - Choosing an algorithm
    - Evaluating performance
    - Hyperparameter tuning

The business intelligence process visualized

Returning to this process visualization. Now we move to data mining

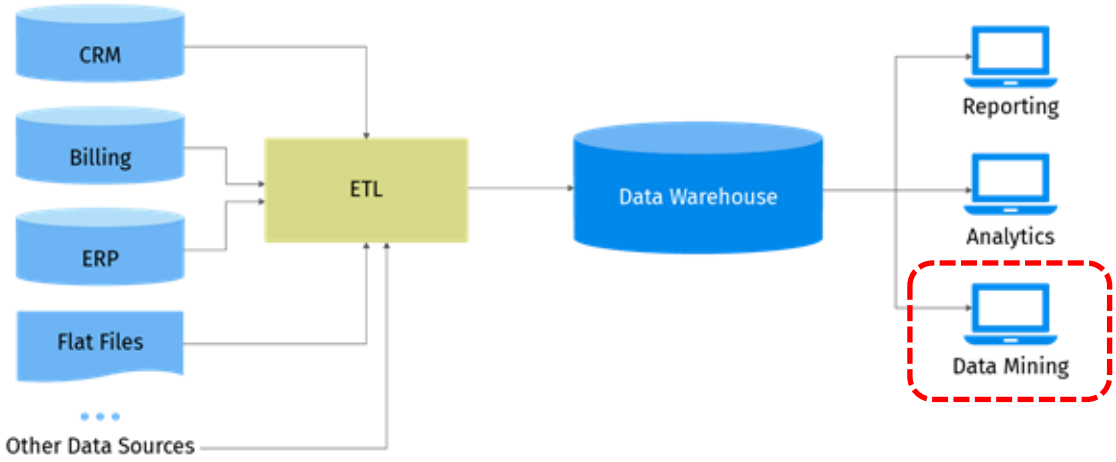


Image from: https://blog.griddynamics.com/edw-performance-comparison/

What is machine learning

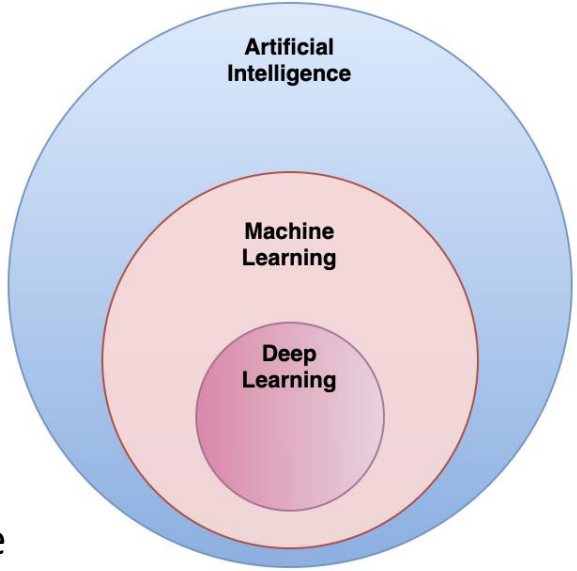
What is machine learning

Machine learning (ML) is the field of study that gives computers the ability to learn without being explicitly programmed

Deep learning (DL) is a subfield of ML which focuses on artificial neural networks

Both ML and DL are considered to be a part of **artificial intelligence** (AI)

Applying ML techniques to discover patterns that were not immediately apparent can be called **data mining**



Traditional programming

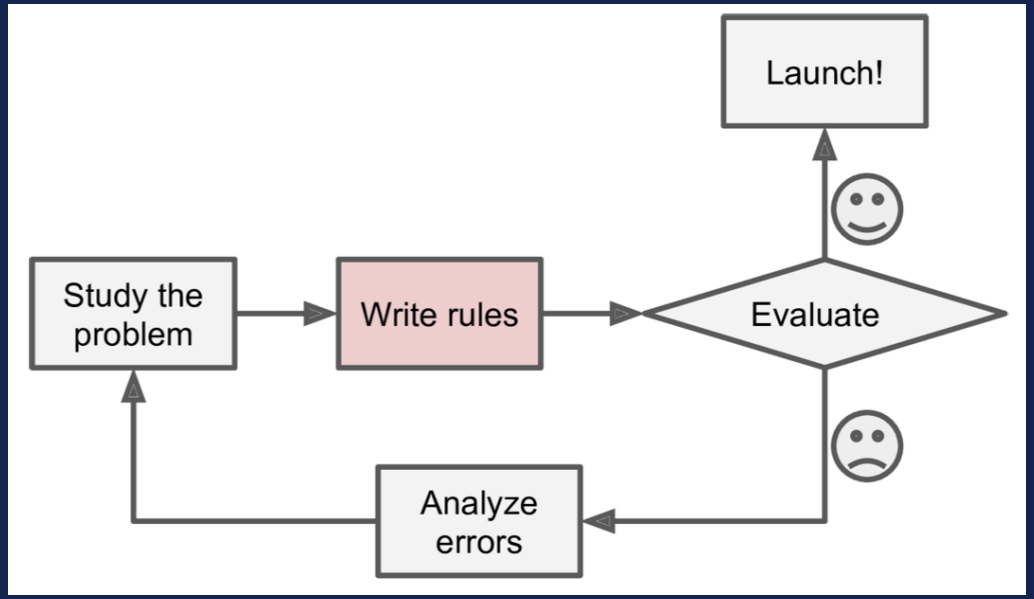


Image from Géron, A. (2019)

Machine learning

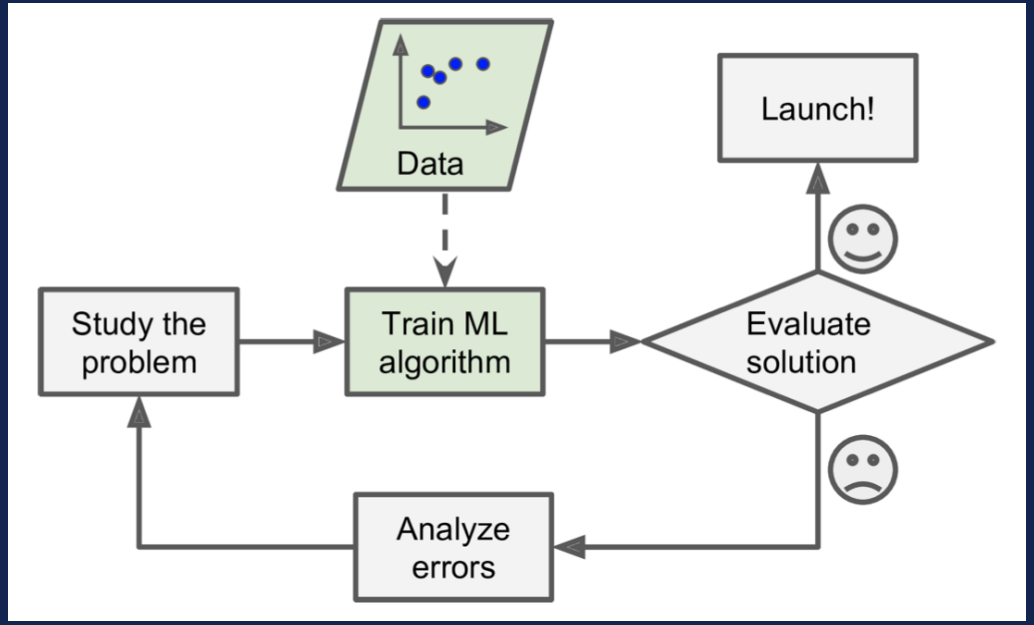


Image from Géron, A. (2019)

Machine learning

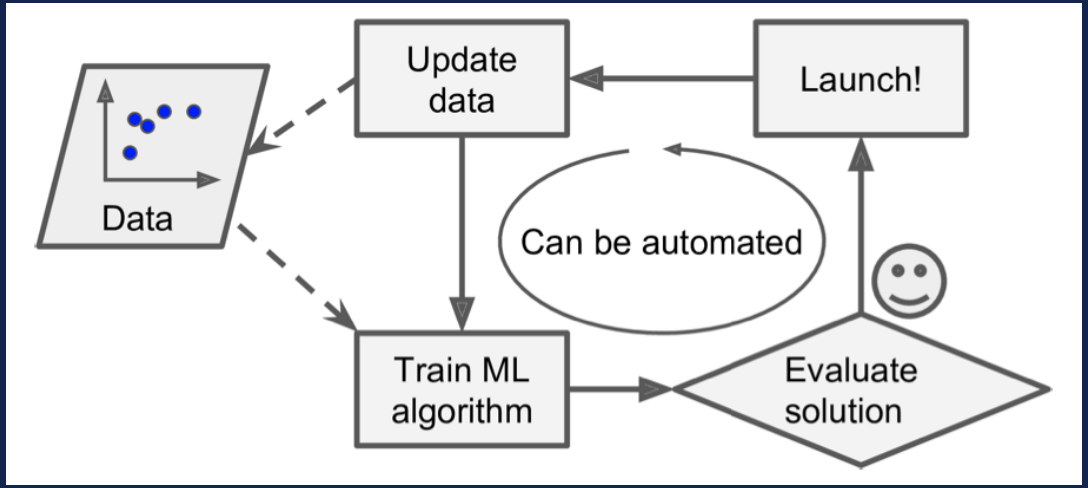


Image from Géron, A. (2019)

Why use machine learning

Creating a rules-based program can be difficult and maintaining it can be time consuming

ML techniques work well in situations where:

- there is data available on the problem
- the problem is in a fluctuating or evolving environment
- the problem involves making predictions or discovering trends

How machine learning works in a nutshell*

1. Collect data and preprocess it
2. Split a dataset into training and test data**
3. Choose an algorithm and set its hyperparameters***
4. Train a model with the training data
5. Evaluate how well the model works with the test data
6. Repeat steps 2-4 until you are satisfied with the results

* This is an example of supervised machine learning
** Why we do this is explained in detail later during this lecture
*** Hyperparameters control the learning process (how the model learns from data)

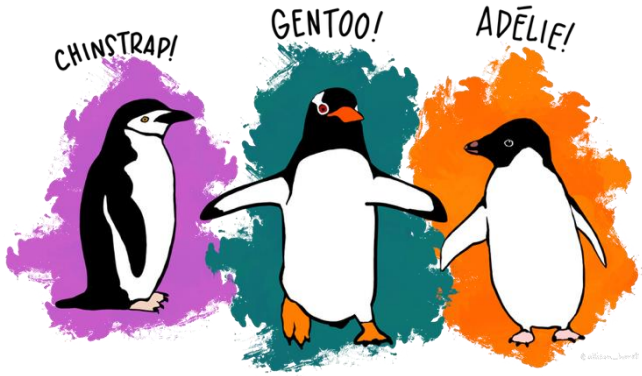
A simple example of a ML project

The goal is to make a program that tells you which species a penguin is based on some characteristics

We use the now familiar Penguins dataset that contains data on over 300 penguins from 3 species. We clean the data and split it to two sets, training data and test data

The next step is to choose an algorithm and then train it with the training data and evaluate its performance on the test set

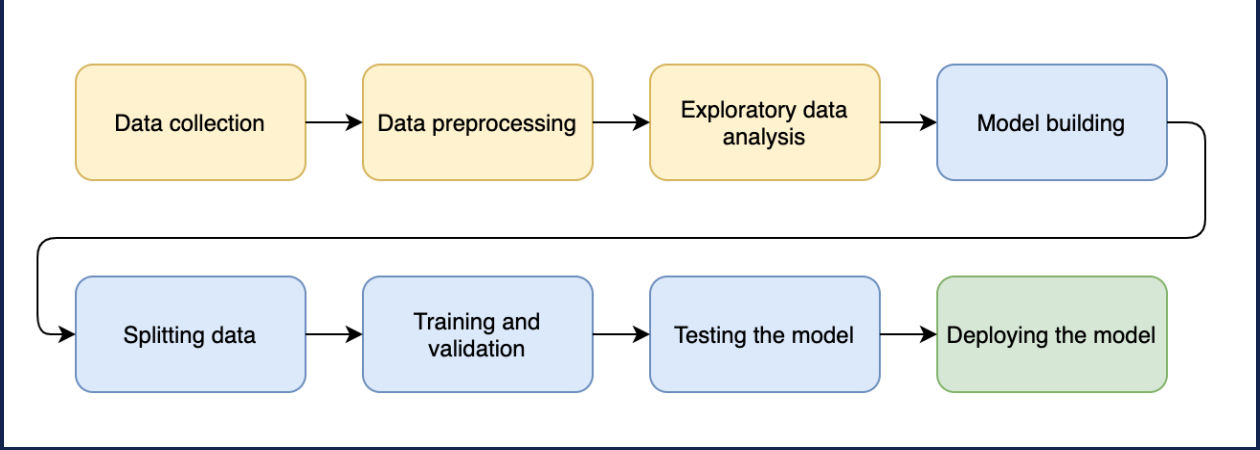
Once we have adjusted and retrained the model and are satisfied with its performance, we can now predict what species new penguin that we come across belongs to



Supervised ML example

Going through *L6_1_classifying_penguings_demo.ipynb*

A more realistic example of a ML project



The yellow boxes represent preparation and blue boxes the development of the ML model

In reality, projects are rarely linear and there will be a lot of going back and forth between the steps

Types of machine learning systems

ML methods can be categorized broadly based on

How they are trained:

- supervised machine learning
- unsupervised machine learning
- reinforcement learning (not covered in this course)

How they work:

- instance-based
- model-based

Supervised machine learning in simple terms

The goal of supervised machine learning is to train a model that can make accurate predictions

To be able to train the model you must have a “labeled” dataset, that has examples of data and a label that you want to train the ML model to predict.

If we want to predict the value of houses for sale, to train the model we need a historical dataset with characteristics of houses and the price that they were sold for

During training, an algorithm learns patterns from the dataset, and we can see with the labels of the training data whether the model is accurately predicting the labels

After training, we could use the model to make predictions on data that does no have labels. E.g., go show the model a house’s characteristics and the model predicts a good value for it

Supervised machine learning

The training algorithm is shown training data that includes labels

E.g., training a model to recognize dog breeds from photos

Examples of algorithms:

- Linear regression
- K-nearest neighbors
- Random forest

Stanford Dogs dataset
(120 breeds)

Classification and regression in supervised learning

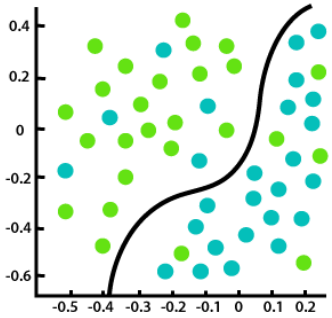
Differ in the types of labels (dependent variables)

Classification:

- Discrete/categorical labels
- Image labeling, sentiment prediction

Regression:

- Continuous labels
- Stock prices, temperature, sales volume



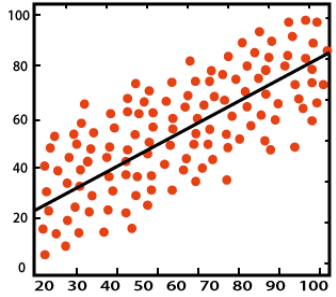


Image from: https://www.javatpoint.com/regression-vs-classification-in-machine-learning

Labeling spam mail with supervised ML

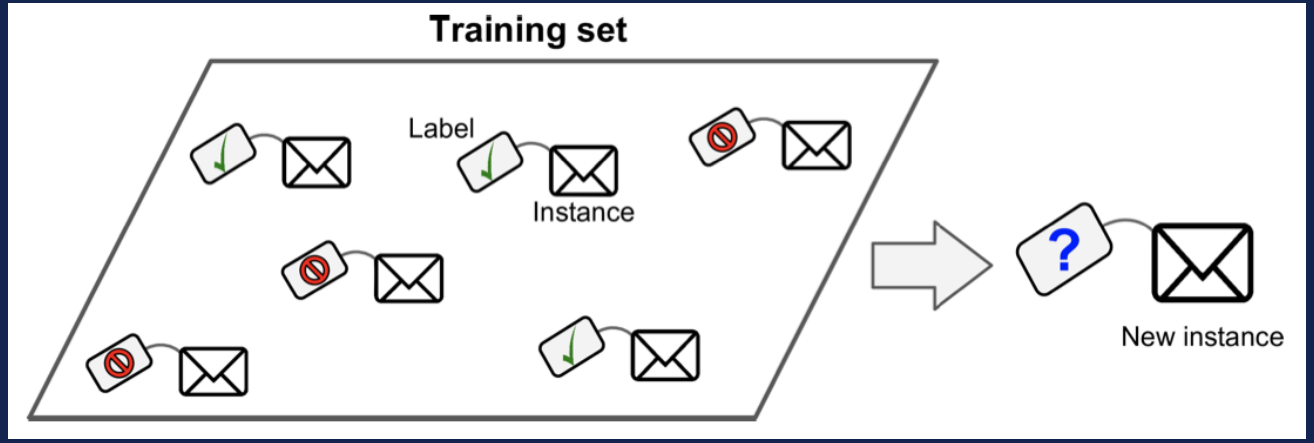


Image from Géron, A. (2019)

Think of an example of a regression problem

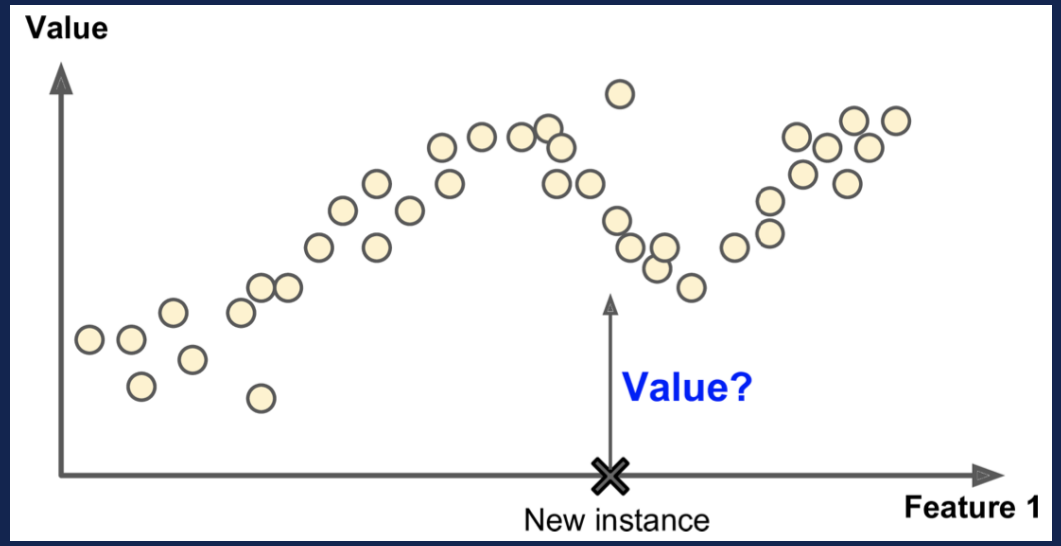


Image from Géron, A. (2019)

Instance-based vs model-based learning

One way to classify supervised machine learning systems is on how they make predictions when shown new data (generalize)

Instance-based learning:

Learns all previous data and compares new data to it and generalizes based on similarity

Model-based learning:

The algorithm builds a model based on the training data and predicts labels according to the model

Instance-based learning

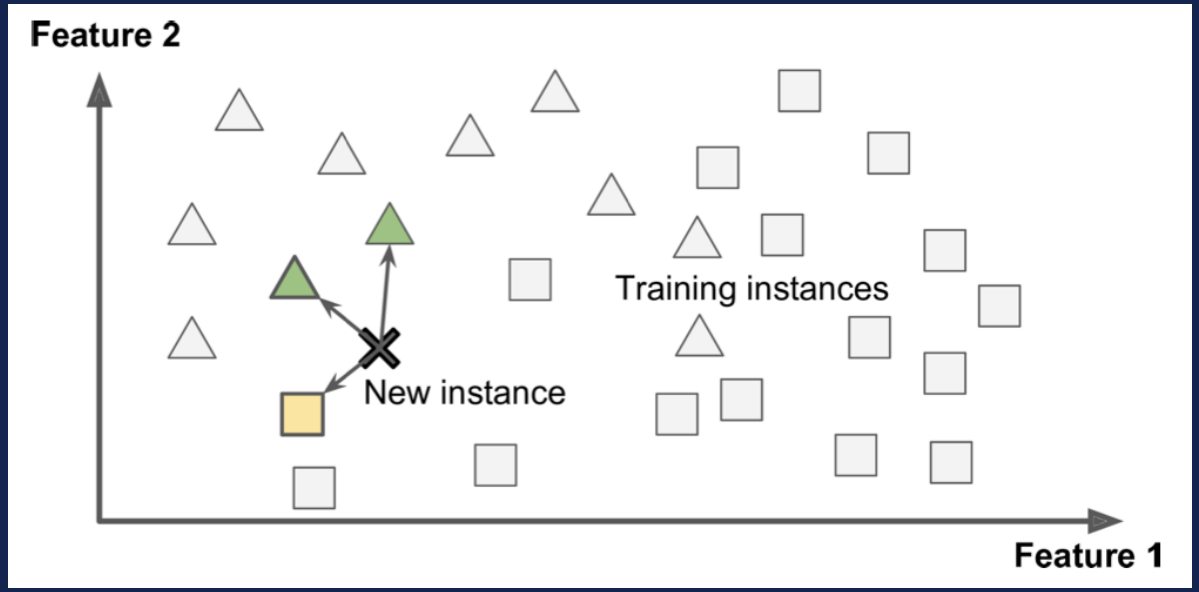


Image from Géron, A. (2019)

Model-based learning

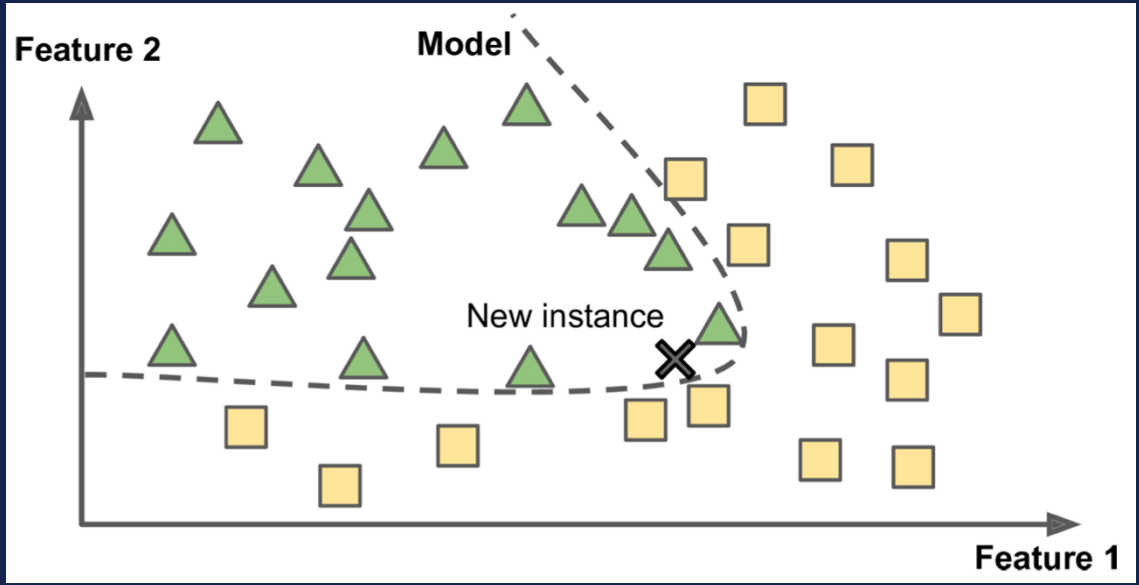


Image from Géron, A. (2019)

A note on terminology: algorithms and models

Sometimes people erroneously use ML terms interchangeably. Note the difference between an algorithm and a model

Algorithm:

An algorithm is a set of procedures that creates a model when trained. E.g., linear regression, random forest

Model:

The model is a specific instance of a fitted (trained) algorithm. E.g., a linear regression model that has been trained to predict house prices in Helsinki

Challenges of machine learning

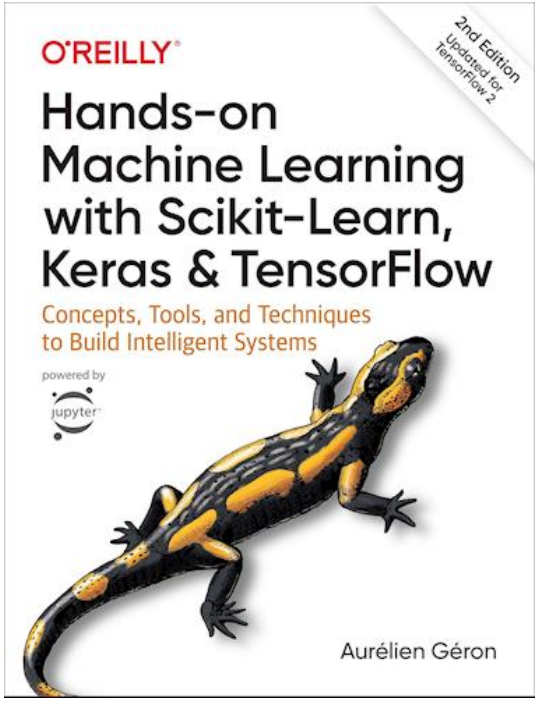
- Not enough training data
- Nonrepresentative training data
- Poor quality data
- Irrelevant features (variables) included in the model
- Overfitting (overgeneralizing) or underfitting

Recommended reading beyond this course

This lecture is loosely based on the book

Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems

If you want to continue studying machine learning specifically after this course, this book and its materials are a great starting point



Preprocessing for machine learning

Preprocessing data for machine learning

For machine learning we will primarily be using the library scikit-learn: https://scikit-learn.org/stable/

scikit-learn does not work with text (strings) and besides the usual data cleaning and preprocessing, as an additional step non-numeric variables (aka features) must be converted to numeric form

Changing categorical values to numerical values

Datasets can contain categorical and ordinal features with text and these need to be converted to numeric form

There are several ways to do this:

- One-hot encoding
- Label encoding
- Ordinal encoding

island
Torgersen
Torgersen
Torgersen
Torgersen
Torgersen

island_Biscoe	island_Dream	island_Torgersen
0	0	1
0	0	1
0	0	1
0	0	1
0	0	1
...	...	...
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0

Dummy variables and one-hot encoding

A dummy variable is a binary variable (0/1)

One-hot encoding transforms each possible value from a categorical variable into a new binary column

island
Torgersen
Torgersen
Torgersen
Torgersen
Torgersen

island_Biscoe	island_Dream	island_Torgersen
0	0	1
0	0	1
0	0	1
0	0	1
0	0	1
...	...	...
1	0	0
1	0	0
1	0	0
1	0	0
1	0	0

```
pd.get_dummies()
```

```
sklearn.preprocessing.OneHotEncoder()
```

Label encoding

Instead of creating columns with binary encoding, each possible value is given a unique integer value

In the Penguins case this means Torgensen = 2, Biscoe = 1, Dream = 0

Note! ML algorithms might misjudge there to be an order or relationship in the data that does not really exist

island
Torgersen
Torgersen
Torgersen
Torgersen
Torgersen

island
2
2
2
2
2
...
0
0
0

```
sklearn.preprocessing.LabelEncoder()
```

Ordinal encoding

Very similar to label encoding, but specifically for maintaining order of the values

Since the penguin dataset has no text variable that is ordinal, the example demonstrates using body_mass_g

body_mass_g
3750.0
3800.0
3250.0
3450.0
3650.0
...

size
30
32
11
18
26
...

```
sklearn.preprocessing.OrdinalEncoder()
```

Scaling and normalizing data

Some learning algorithms are sensitive to the scale differences in variables

E.g. one variable is in the range of 0-1 while another 100-10000

To mitigate such issues data can be

- scaled (e.g. from 0 to 1)
- standardized (data points are expressed as σ from the mean)
- log transformed

```
sklearn.preprocessing.MinMaxScaler()
```

```
sklearn.preprocessing.StandardScaler()
```

Train test split

Train test split

An important aspect in machine learning is that we want to train a model on some data, and test it on a completely unseen separate set of data

After completing pre-processing, the dataset is split into a training set and test set, which should have similar characteristics (balanced distribution of different categorical data)

Train, validation and test sets

In supervised ML datasets are divided into 2 or 3 subsets:

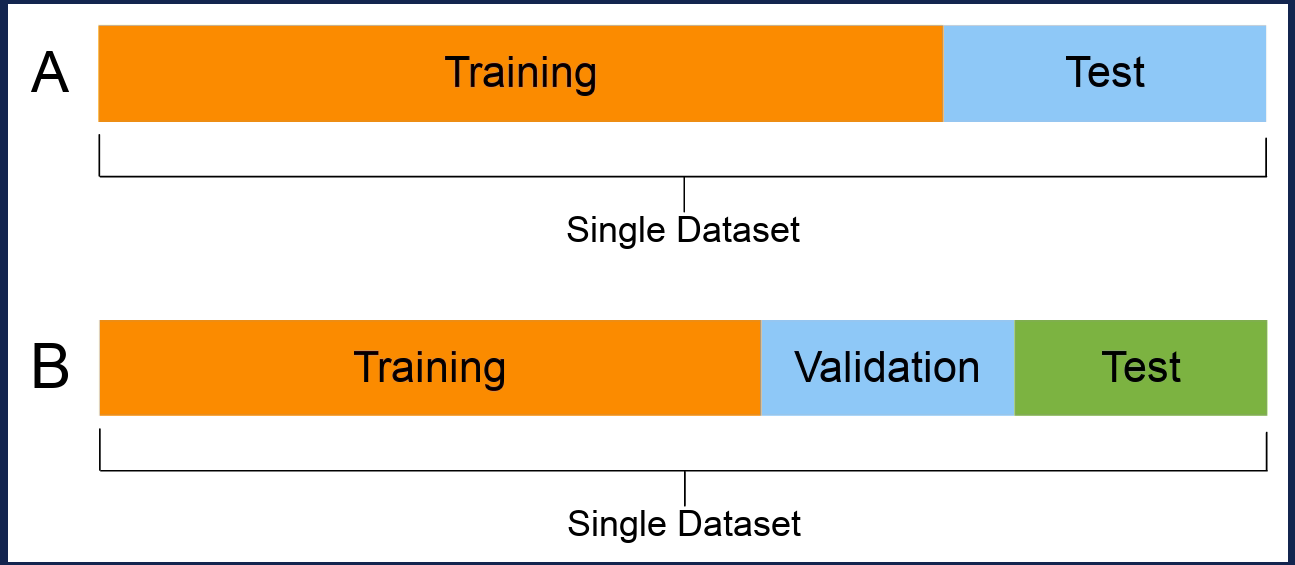
- Training set, to train the model
- Test set, to confirm that the model works

Or

- Training set, to train the model
- Validation set, to test the set while still adjusting the model after each training and hyperparameter change
- Test set, to confirm that the model works with unseen data

```
X_train, X_test, y_train, y_test = train_test_split(…)
```

Train, validation and test sets



Why use a validation and separate test set?

Fine-tuning a ML model is an iterative process where you want to adjust the algorithm to perform better so that it performs well on both the training data and test set

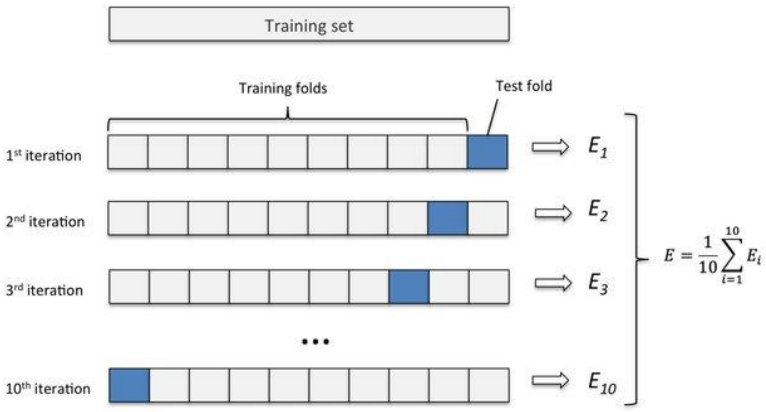
To ensure your model can generalize, it is recommended to evaluate the model on a validation set instead of the test set during hyperparameter tuning

The test set (aka holdout set) is used only on the final model to get an unbiased evaluation of the model's performance and to ensure you haven't overfitted the model to perform well on the testing data

Cross-validation as an alternative

A less biased or less optimistic estimate of the model than train/test split

Typically implemented as K-folds cross validation, where k is the number of splits




```
KFold(n_splits=10, random_state=42, shuffle=False)
```

Image from Introduction to Support Vector Machines and Kernel Methods, Ashfaque and Iqbal, 2019

Cross-validation

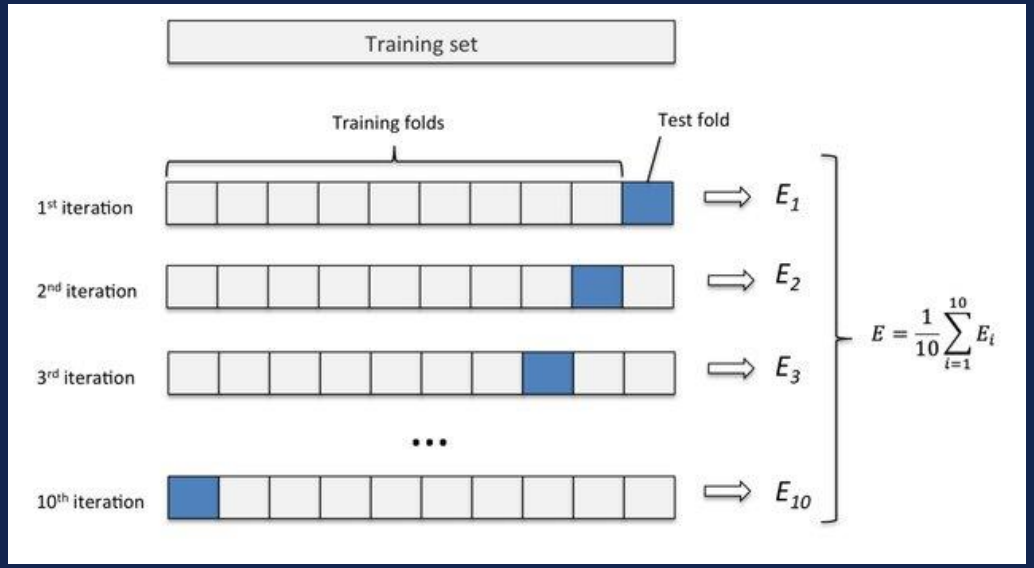


Image from Ashfaque and Iqbal (2019)

Choosing an algorithm

Choosing an algorithm

There are many different learning algorithms to choose from and this lecture covers only a few basic examples

Some algorithms are better suited for specific types of datasets based on e.g. how the data is distributed, whether the problem is classification (binary or multiclass) or regression and so on

The course book has good in-depth explanations of random forest, and support vector machines

Terminology: parameters and hyperparameters

ML algorithms have hyperparameters and models have parameters

Hyperparameters are set by the user to control the algorithm and they define how it learns from the data

Parameters are the internal variables within a model that are adjusted automatically as you train a ML model

K-nearest neighbors

Abbreviated as KNN

Labels new data based on similarity to examples in training data

Nonparametric, instance-based: does not “learn a model”

Examples are classified using a distance function such as Euclidian or Manhattan shown below respectively

$$\sqrt{\sum_{i=1}^{n}(a_i - b_i)^2}$$

$$\sum_{i=1}^{n}|a_i - b_i|$$

for two points a = (a_1, a_2, … a_n) and b= (b_1, b_2, … b_n)

K-nearest neighbors

Given a training dataset X and a new instance x_{new}

Find k points in X that are closest to x_{new}

Using the selected distance measure:

Predict label for x_{new}

- Classification: majority vote among the k nearest neighbors
- Regression: mean of the k nearest neighbors

`sklearn.neighbors.KNeighborsClassifier`

K-nearest neighbors

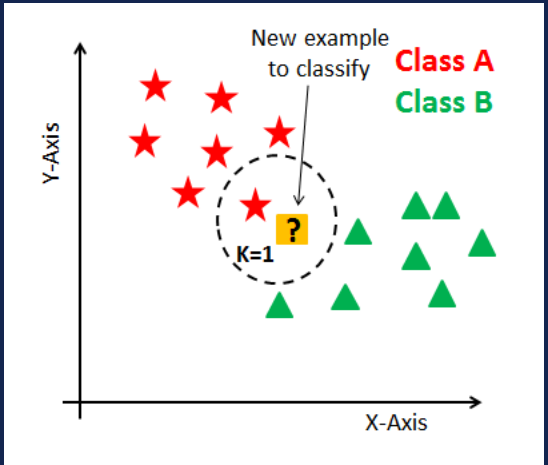


Image from Datacamp

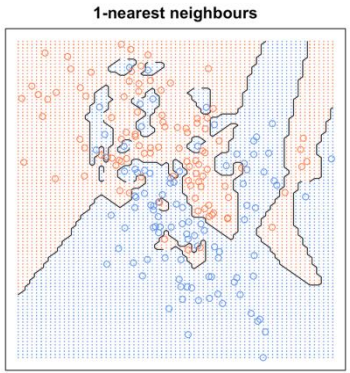
Choosing K

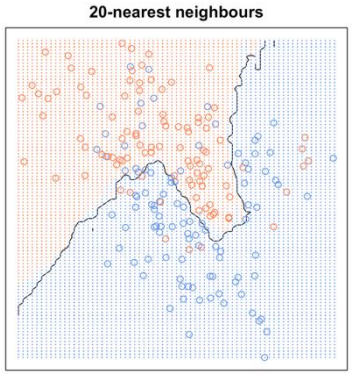
Larger *k*:

- Suppresses noise
- Makes classification boundaries smoother
- Less overfitting

Some approaches:

- *k* = sqrt(n)
- Loop over different values of *k* and compare errors
- Use an odd *k*





Images from https://kevinzakka.github.io/2016/07/13/k-nearest-neighbor/

Strengths and weaknesses of KNN

Strengths:

- Easy to interpret
- Easy to implement

Weaknesses:

- All features have equal importance (unlike e.g. decision trees)
- With very large datasets computing distances between datapoints is takes too much time

Things to consider when using KNN

Features need to be scaled:

- A feature with a big scale can dominate all the distances
- A feature with a small scale would get neglected

The "Curse of dimensionality":

- problems with high-dimensional spaces (e.g. more than 5 features)
- volume of space grows exponentially with dimension
- need exponentially more points to 'fill' a high-dimensional volume or you might not have any training points "near" a test point

KNN demo

Going through *L6_2_knn.ipynb*

Performance measures (classification)

Performance measures

Alternatively performance metrics

Used to evaluate how well your supervised ML model works

Different measures will be used depending on the ML model and application

Classification metrics

Describes how many predictions your model got right and wrong and in which class

It is generally a good idea to use multiple metrics

Let’s look at an example where you use only accuracy:

This fraud detection model has 99% accuracy!*

The model was trained with 10000 rows of data, with 100 examples of fraud and 9900 examples of legitimate actions

Confusion matrix

		Predicted	
		Negative *(class 0)*	**Positive** *(class 1)*
Actual	**Negative** *(class 0)*	True negative TN	False positive FP
	Positive *(class 1)*	False negative FN	True positive TP

```
confusion_matrix(y_true, y_pred)
```

Confusion matrix with the example's numbers

		Predicted	
		Negative *(Not fraud)*	**Positive** *(Fraud)*
Actual	**Negative** *(Not fraud)*	9900	0
	Positive *(Fraud)*	100	0

```
confusion_matrix(y_true, y_pred)
```

This fraud detection model has 99% accuracy!*

The model was trained with 10000 rows of data, with 100 examples of fraud and 9900 examples of legitimate actions

Going through the example

One metric and especially accuracy alone is bad

If the classes are imbalanced, high accuracy can be achieved with just predicting everything to belong to the largest class

In the example accuracy was seemingly good although the model was bad and didn’t detect cases of fraud at all

Even with a balanced dataset it is always a good idea to measure and report the performance in multiple ways

Precision, recall, accuracy and F1

$$\text{Accuracy} = \frac{\text{TN} + \text{TP}}{\text{TN} + \text{FP} + \text{FN} + \text{TP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

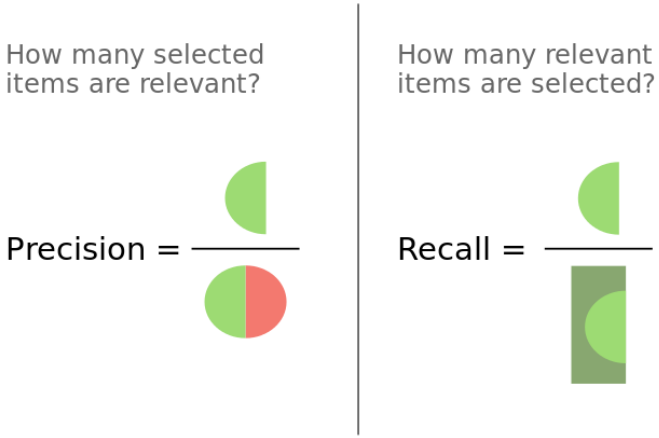
$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

		Predicted	
		Negative *(class 0)*	**Positive** *(class 1)*
Actual	**Negative** *(class 0)*	True negative TN	False positive FP
	Positive *(class 1)*	False negative FN	True positive TP

Recall

Precision

Precision and recall



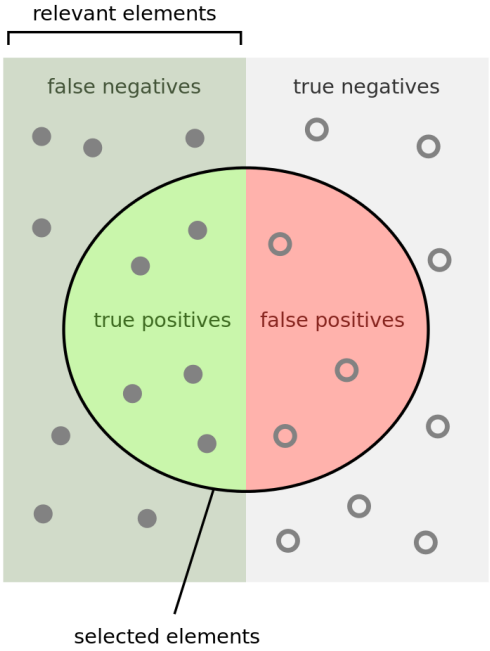


Image from: https://en.wikipedia.org/wiki/F-score#/media/File:Precisionrecall.svg

The ROC Curve and AUC

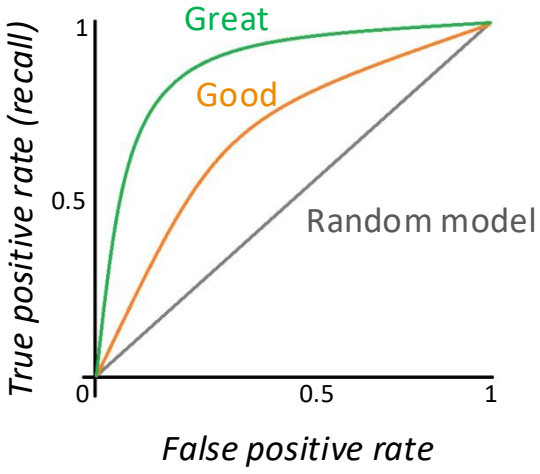
ROC Curve

Receiver operating characteristic curve

Plots the true positive rate (recall) against the false positive rate

AUC

Area under the curve, a way to measure how good the model performs with 1 being perfect



Linear regression

Linear regression

1. One of the common methods used for regression problems is the **linear regression**, which can be implemented using several different algorithms

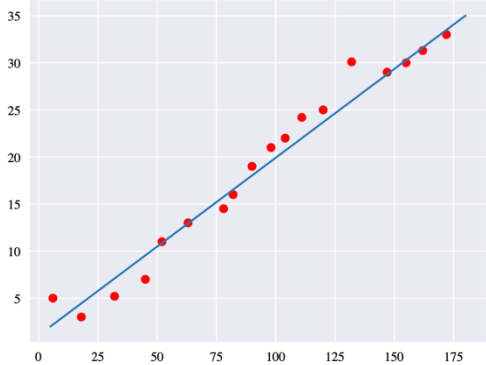
2. Linear regression models are parametric models or in other words an example of model-based learning

3. Note that notation varies a lot depending on the source

Linear regression in ML

1. In machine learning the convention is to write it like this:
2.
3. $y' = b + w_1 x_1$

4. ***y'*** is the predicted label (a desired output)
5. ***b*** is the bias (y-intercept)
6. $\boldsymbol{w_1}$ is the weight of feature 1 (slope)
7. $\boldsymbol{x_1}$ is the feature (input)



Linear regression in ML

Usually there will be multiple features in model and with 3 features it would look like this:

$$y' = b + w_1 x_1 + w_2 x_2 + w_3 x_3$$

The ML algorithm for linear regression attempts to find values for b and w_i that minimizes the loss (e.g. mean squared error)

```
sklearn.linear_model.LinearRegression()
```

What is happening during training

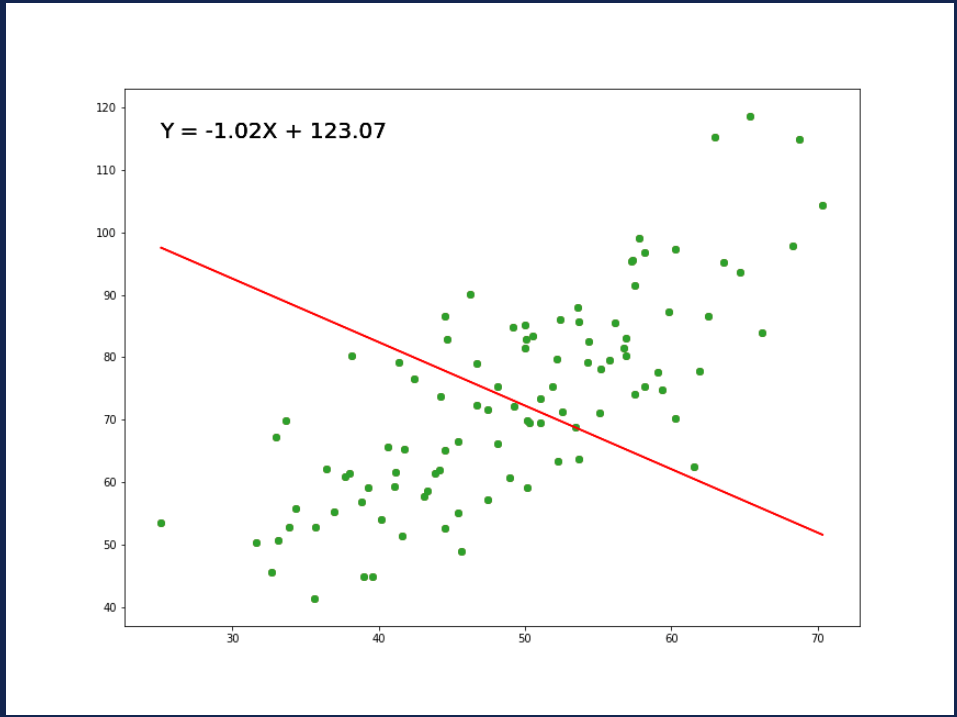


Image from this blog post

Strengths and weaknesses of linear regression

Advantages:

- Easy to interpret
- Easy to implement

Disadvantages:

- Performs poorly when there are non-linear relationships
- Sensitive to outliers

Performance measures (regression)

Regression metrics

Mean squared error (MSE):

$$\mathrm{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_{\mathrm{pred}} - y_{\mathrm{true}})^2$$

Root mean squared error (RMSE):

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

1. **Mean absolute error** (MAE):

$$\mathrm{MAE} = \frac{1}{m}\sum_{i=1}^{m}|y_{\mathrm{pred}} - y_{\mathrm{true}}|$$

2. Which one to use? Depends!

- For most tasks MSE/RMSE works well
- MAE is less sensitive to outliers than MSE/RMSE

Loss in linear regression

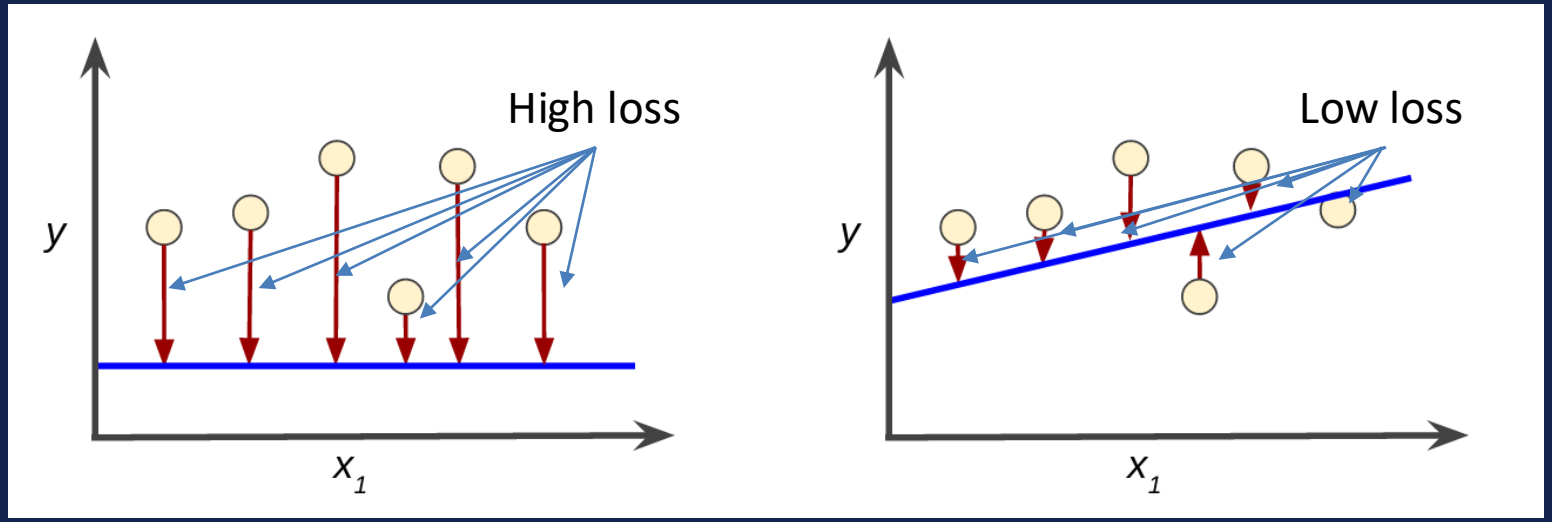


Image from Google

Linear regression demo

Going through *L6_3_linear_regression.ipynb*

Other notable ML algorithms

Other notable ML algorithms

For your projects and general knowledge, I recommend reading about the following algorithms e.g. by reading Wikipedia and documentation:

- Random Forest
- XGBoost
- Support Vector Machine (SVM)

Hyperparameter tuning

Hyperparameter tuning

Manually tuning and trying out different values for hyperparameters is tedious

Grid search and randomized search can be used to go through different combinations of hyperparameter values

The goal is to find the hyperparameter values that give the model optimal performance

Grid search

Grid search systematically evaluated each combination of values for the hyperparameters

Grid search illustrated for 2 hyperparameters of the SVM algorithm: Alpha and C

The values in the matrix are the performance of the model with each hyperparameter combination

5 different values for c

C				
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675
	0.1	0.2	0.3	0.4

Alpha

4 different values for alpha

```
sklearn.model_selection.GridSearchCV()
```

Image from Datacamp

Randomized search

Grid search can be impractical when training times are long and the search space is large

Randomized search evaluates a given number of random combinations

```
sklearn.model_selection.RandomizedSearchCV()
```

Conclusion

Conclusion

This lecture provided merely a glimpse into the world of machine learning

If you are interested in learning more about machine learning, I recommend checking out the previously mentioned book Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow by Aurélien Géron. A pdf version can be found by Googling ☺

Questions, comments suggestions?

References

- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media.